

```

        for (Int j=0;j<n;j++) irank[indx[j]] = j;
    }

};

template<class T>
void Indexx::index(const T *arr, Int nn)
Indexes an array arr[0..nn-1], i.e., resizes and sets indx[0..nn-1] such that arr[indx[j]]
is in ascending order for j = 0, 1, ..., nn-1. Also sets member value n. The input array arr is
not changed.
{
    const Int M=7,NSTACK=64;
    Int i,indxt,ir,j,k,jstack=-1,l=0;
    T a;
    VecInt istack(NSTACK);
    n = nn;
    indx.resize(n);
    ir=n-1;
    for (j=0;j<n;j++) indx[j]=j;
    for (;;) {
        if (ir-l < M) {
            for (j=l+1;j<=ir;j++) {
                indxt=indx[j];
                a=arr[indxt];
                for (i=j-1;i>=l;i--) {
                    if (arr[indx[i]] <= a) break;
                    indx[i+1]=indx[i];
                }
                indx[i+1]=indxt;
            }
            if (jstack < 0) break;
            ir=istack[jstack--];
            l=istack[jstack--];
        } else {
            k=(l+ir) >> 1;
            SWAP(indx[k],indx[l+1]);
            if (arr[indx[l]] > arr[indx[ir]]) {
                SWAP(indx[l],indx[ir]);
            }
            if (arr[indx[l+1]] > arr[indx[ir]]) {
                SWAP(indx[l+1],indx[ir]);
            }
            if (arr[indx[l]] > arr[indx[l+1]]) {
                SWAP(indx[l],indx[l+1]);
            }
            i=l+1;
            j=ir;
            indxt=indx[l+1];
            a=arr[indxt];
            for (;;) {
                do i++; while (arr[indx[i]] < a);
                do j--; while (arr[indx[j]] > a);
                if (j < i) break;
                SWAP(indx[i],indx[j]);
            }
            indx[l+1]=indx[j];
            indx[j]=indxt;
            jstack += 2;
            if (jstack >= NSTACK) throw("NSTACK too small in index.");
            if (ir-i+1 >= j-1) {
                istack[jstack]=ir;
                istack[jstack-1]=i;
                ir=j-1;
            } else {

```

```

        istack[jstack]=j-1;
        istack[jstack-1]=1;
        l=i;
    }
}
}
}

```

A typical use of `Indexx` might be to rearrange three vectors (not necessarily of the same type) into the sorted order defined by one of them:

```

Indexx arrindex(arr);
arrindex.sort(arr);
arrindex.sort(brr);
arrindex.sort(crr);

```

The generalization to any other number of arrays is obvious.

The `Indexx` object also provides a method `el` (for “element”) to access any vector in `arr`-sorted order without actually modifying that vector (or, for that matter, `arr`). In other words, after we index `arr`, say by

```
Indexx arrindex(arr);
```

we can address an element in `brr` that corresponds to the j th element of a *virtually sorted* `arr` as simply `arrindex.el(brr, j)`. Neither `arr` nor `brr` are altered from their original state. `el` is provided in two versions, so that it can be both an l-value (on the left-hand side of an assignment) and an r-value (in an expression).

As an aside, the reason that the internal workhorse `index` uses a pointer, not a vector, for its argument is so that it can be used (purists would say misused) in other situations, such as indexing one row in a matrix. That is also the reason for providing an additional, empty, constructor. If you want to index `nn` consecutive elements sitting around somewhere, pointed to by `ptr`, you write

```
Indexx myhack;
myhack.index(ptr, nn);
```

A *rank table* is different from an index table. A rank table’s j th entry gives the rank of the j th element of the original array of keys, ranging from 0 (if that element was the smallest) to $N - 1$ (if that element was the largest). One can easily construct a rank table from an index table. Indeed, you might already have noticed the method `rank` in `Indexx` that returns just such a table, stored as a vector.

Figure 8.4.1 summarizes the concepts discussed in this section.

8.5 Selecting the *M*th Largest

Selection is sorting’s austere sister. (Say *that* five times quickly!) Where sorting demands the rearrangement of an entire data array, selection politely asks for a single returned value: What is the k th smallest (or, equivalently, the $m = N - 1 - k$ th largest) element out of N elements? (In this convention, used throughout this section, k takes on values $k = 0, 1, \dots, N - 1$, so $k = 0$ is the smallest array element and $k = N - 1$ the largest.) The fastest methods for selection do, unfortunately, rearrange the array for their own computational purposes, typically putting all smaller elements